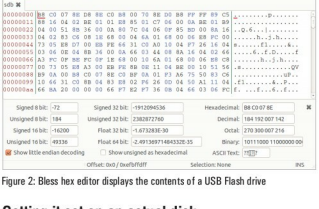

Figure 2: Bless hex editor displays the contents of a USB Flash drive

Getting it set on an actual disk

Let's get our code written onto the bootsector of a USB Flash drive. First, find the drive name of the USB Flash drive. If you have just a single SATA hard disk installed, the next drive you connect (here, the USB Flash drive) would be called */dev/sdb*. To verify this, launch the *Disk Utility* or try the command *lsblk* after connecting the drive.

⚠ Caution:

Incorrect identification of drive names can cause unexpected data loss. Also, take a backup of important files before you start experimenting with a disk.

We are not performing a simple copy-paste. So it is better to unmount the drive (in case it got mounted automatically) by using the following command:

```
sudo umount /dev/sdb
```

Now, we can use the following command to copy the bootsector. But wait! Don't even think about executing it until you complete the next paragraph.

```
sudo dd if=hello.bin of=/dev/sdb
```

This command will copy the data in the file *hello.bin* to the Flash drive, directly (i.e., not as a file). To make it useful again, you'll have to format it. Alternatively, the following commands copy the necessary parts only, leaving all other bytes intact.

```
sudo dd bs=446 count=1 conv=notrunc if=hello.in of=/dev/sdb
sudo dd bs=1 count=2 seek=510 skip=510 conv=notrunc if=hello.
bin of=/dev/sdb
```

The first line copies the boot code and the second line copies the boot signature bytes (0x55, 0xaa). *bs* means the block size is to be considered while copying; *count* is the number of blocks to copy; *seek* is the number of blocks to skip at the start of the output; and *skip* is the number of blocks to skip at the start of the input. *conv=notrunc* ensures that the output file is not truncated (i.e., all other sectors in the USB Flash drive are preserved).

Now let's test it on the real BIOS. Connect the USB drive and restart the computer. If USB booting is enabled and given high preference, the system should directly load your boot code. If it doesn't, restart again and enter the BIOS settings. Ensure the following, save the settings and restart again:

- USB booting is enabled and given preference over other drives like HDD.
- Legacy booting is enabled and is preferred over UEFI.

Now you should see your 'Hello World' program running. When you've finished enjoying your own bootable program, just press the power button on the tower to shut down, or use *Ctrl + Alt + Del* to restart.

Using a hex editor

A hex editor is a program that lets you view/edit the contents of any computer file as a plain stream of bytes, and it is usually represented in hexadecimal values. While developing bootable programs, we can use a hex editor to review the internal byte patterns in a disk image or pseudo-files like */dev/sdb*. In our case, for example, you can use a hex editor to ensure that we haven't exceeded the limit of 446 bytes for our boot code.

The popular command *hexdump* might be already available on your GNU/Linux system. However, I recommend installing the package *bless*, which provides the easy and full-featured Bless hex editor.

After installation, launch Bless from the menu or by using the command *bless*. If you are not the root user, you might not be able to open system files like */dev/sdb* (which represents the hard disk). You'll need the assistance of commands like *su*, *sudo* or *gksudo*. For example:

```
sudo bless /dev/sdb
gksudo bless /dev/sdb
```

⚠ Caution:

Never open actual disks (especially your primary hard disk) if you are not sure what you are doing. Editing files like */dev/sda* can even break the booting, OS and the partition table. However, trying disk images (e.g., *.img* and *.iso* files) is safe if you have their backup copies.

Another example: A simple typewriter

When we run *hello.bin* (our first example), the keyboard does not respond. Given below is the code for a simple typewriter, which displays whatever you type.

```
mov ax, 0x7c0
mov ds, ax
```